

Company-Level DSA Practice Questions

Topics: Linear Search • Binary Search • Bubble Sort • Selection Sort • Insertion Sort • Merge Sort

Level: Company / Placement Coding Round

1. Linear Search — First Non-Repeating Element

A company stores transaction IDs in an integer array. Some transaction IDs may occur multiple times. Find the **first transaction ID that occurs exactly once** and return its index.

Requirements:

- Use Linear Search.
- Do not sort the array.
- Do not use another array.
- Do not use hashing/frequency maps.
- Return the index of the first non-repeating element.
- If no such element exists, return -1.

Input: arr = [4, 5, 1, 2, 1, 4, 5, 7, 2]

Output: 7

2. Binary Search — First and Last Position

A company maintains employee IDs in sorted order, but the same ID can occur multiple times. Given a sorted array and a target employee ID, find the **first and last occurrence** of that ID.

Requirements:

- Must use Binary Search.
- Do not scan the entire array after finding the target.
- Time complexity must be $O(\log N)$.
- Return [-1, -1] if the ID does not exist.
- Use two binary searches: one for first occurrence and one for last occurrence.

Input: arr = [1, 2, 2, 2, 3, 4, 5], target = 2

Output: [1, 3]

3. Bubble Sort — Count Passes and Swaps

An application receives an array of employee ratings. Sort the ratings in ascending order using **Bubble Sort**. The company wants to monitor the number of operations performed.

Requirements:

- Use Bubble Sort only.
- Count total swaps.
- Print the array after every pass.
- Stop immediately when a complete pass performs zero swaps.
- Print the number of passes and total swaps.

Input: [5, 1, 4, 2, 8]
Output: Sorted Array: [1, 2, 4, 5, 8]
Passes: ?
Swaps: ?

4. Selection Sort — Kth Smallest Distinct Element

A company has employee salary values. Find the **Kth smallest distinct salary**.

Requirements:

- Use Selection Sort logic.
- Duplicate salaries count only once.
- Do not use another array.
- Do not use hashing.
- Do not completely sort the array if unnecessary.
- If fewer than K distinct salaries exist, return -1.

Input: arr = [50000, 30000, 40000, 30000, 60000, 50000, 20000], K = 3
Output: 40000

5. Insertion Sort — Stable Record Sorting

A company receives student records based on marks. Sort the students by marks in ascending order while preserving the original order of students having equal marks.

Requirements:

- Use Insertion Sort.
- Sort both marks and corresponding student IDs.
- Equal marks must retain their original order.
- Do not use another array.
- Do not use built-in sorting functions.

Marks: [80, 70, 80, 60, 70]
IDs: [101, 102, 103, 104, 105]

Expected Marks: [60, 70, 70, 80, 80]
Expected IDs: [104, 102, 105, 101, 103]

6. Merge Sort — Count Inversions

An e-commerce company records product prices in the order products were received. An inversion exists when $i < j$ and $arr[i] > arr[j]$. Find the total number of inversions.

Requirements:

- Must use Merge Sort.
- Time complexity must be $O(N \log N)$.
- Do not use nested loops to count every pair.
- Duplicate values are not considered inversions.
- The answer can be very large, so use an appropriate data type.

Input: arr = [8, 4, 2, 1]

Output: 6

Inversions:

(8,4), (8,2), (8,1), (4,2), (4,1), (2,1)

Priority for placement preparation: Q2 (Binary Search), Q4 (Selection Sort), and Q6 (Merge Sort / Inversion Count).